

Beachlorarbeit

Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen Welten-Generation

Maarten Behn

CGVR Universität Bremen

08.09.2026

Kern Idee

1. Kern Idee

—



1. Kern Idee

- Angenommen es gibt eine prozedurale Welt
- Dann gibt es im Code des Spiels ein Haufen Funktionen, die eine neue Welt erzeugen kann bzw. neue Bereiche errechnen.

Bild von Minecraft

- Man kann diesen Code auch als eine Funktion verstehen
- Input: alle Parameter der Welt
- Eine Spielwelt
- Da viel Zufälligkeit genutzt wird, gibt es eine Menge an Welten, die generiert werden kann.

Related Work

2. Related Work

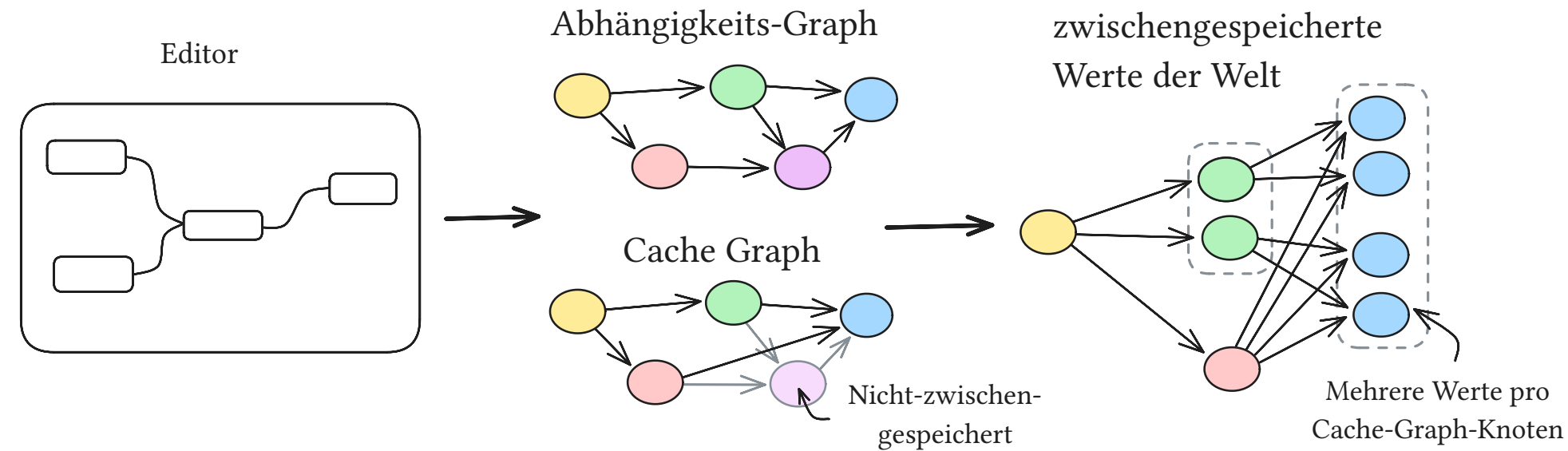
—

Was ich gebaut habe

3. Was ich gebaut habe

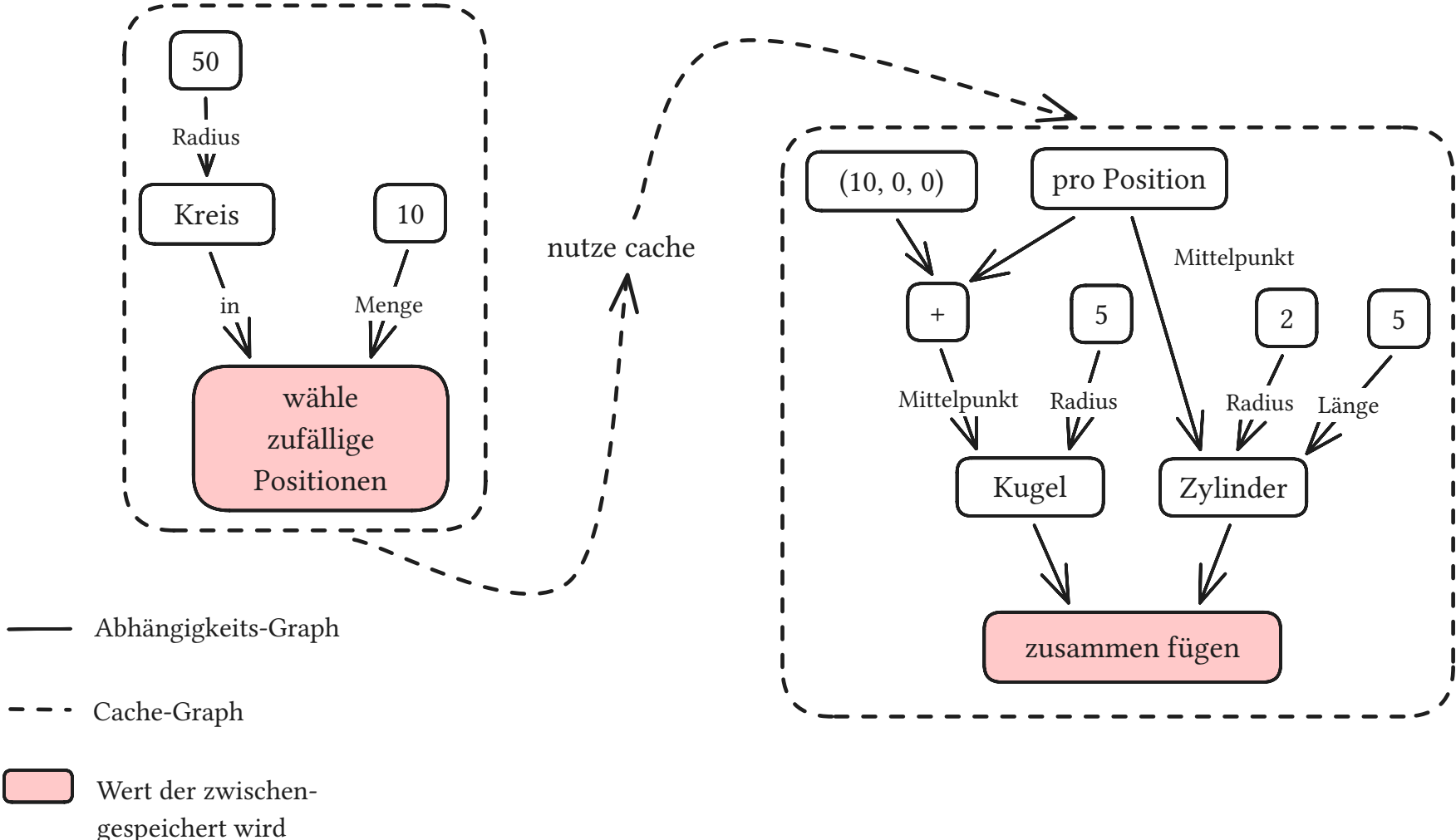
—

- Bild vom Programm mit Bestanteilen erklärt



3. Was ich gebaut habe

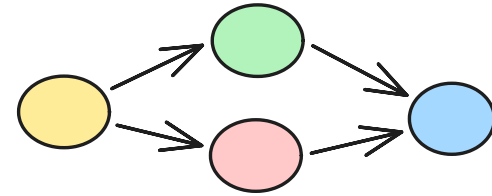
- Kurzes Video
 - Zeigt wie man im Editor eine Welt erstellt und sich diese live im Renderer angezeigt wird.
- Editor, Template, Generator
- Bild der beiden Graphen
- Einer der den Algorithmus darstellt
- Eine Kante zu jedem Wert den es als Input nutzt (abhängt)
- Und einer der nur ein Knoten pro gecachten Knoten in dem Algorithmus enthält.



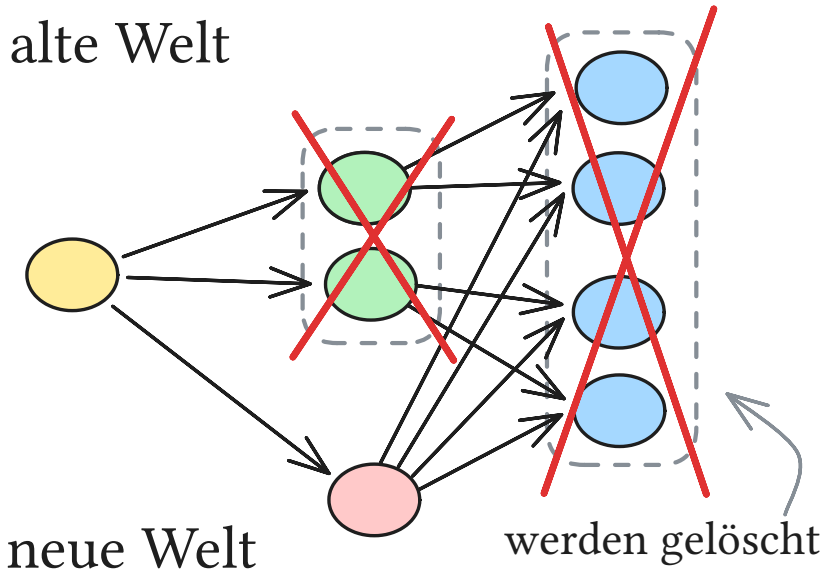
Templates vergleichen

3. Was ich gebaut habe — 3.2 Templates vergleichen

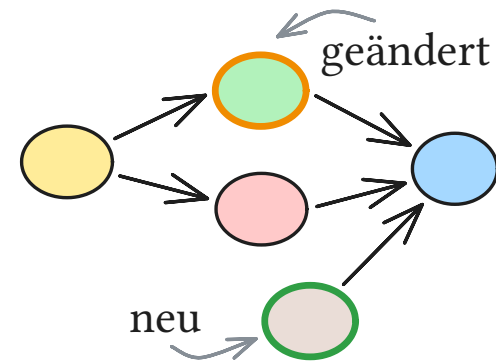
altes Template



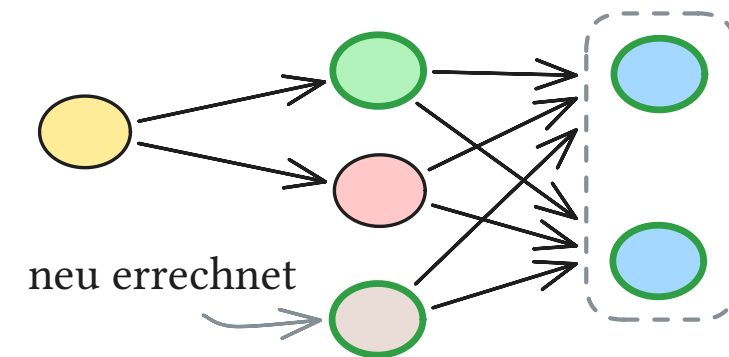
alte Welt



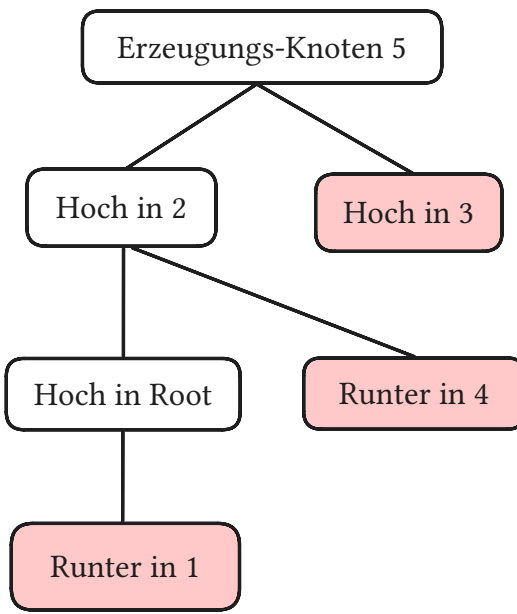
neues Template



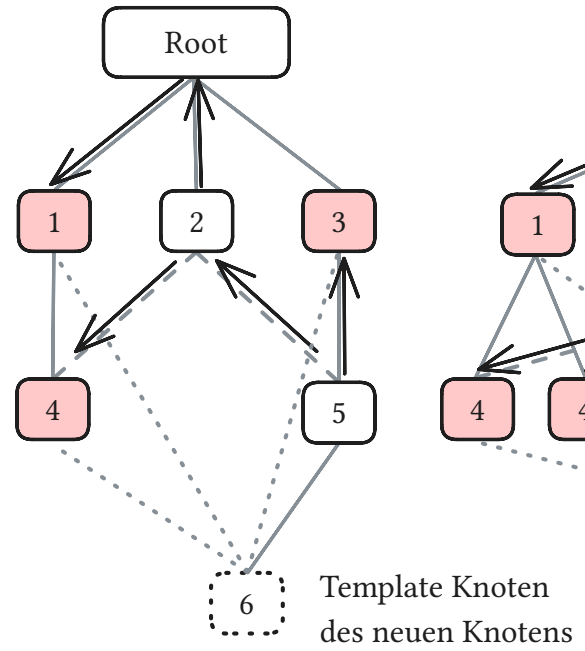
neue Welt



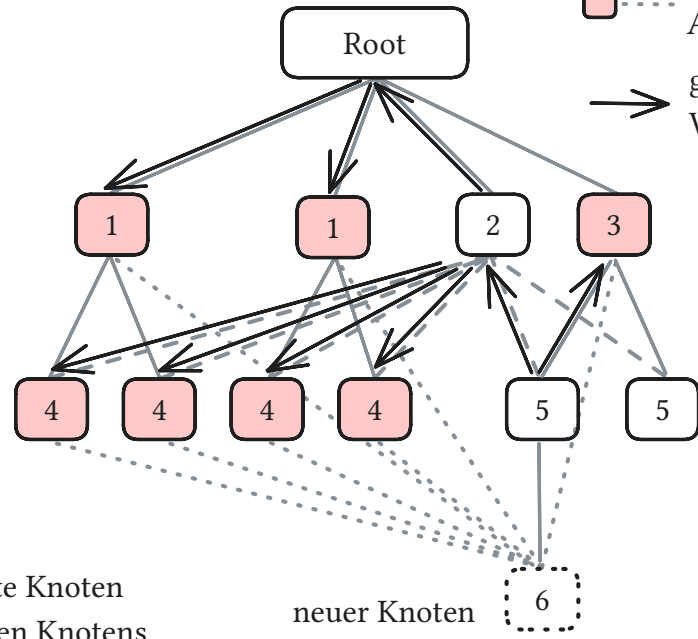
Baum an relativen Schritten



Template



Generator

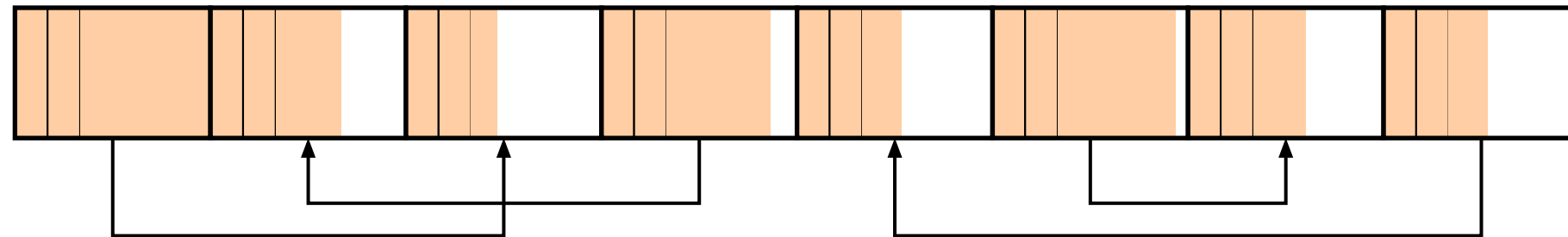


- erstellt
- - - hängt ab
- gesuchte Abhängigkeit
- genommene Wege

```
enum TemplateValue {  
    Number(NumberValue),  
    Position(PositionValue),  
    Volume(VolumeValue),  
    ...  
}
```

```
enum PositionValue {  
    Add((PosIndex, PosIndex)),  
    Sub((PosIndex, PosIndex)),  
    Numbers([NumberIndex; 3]),  
    ...  
}
```

```
let values: Vec<TemplateValue>
```



3. Was ich gebaut habe

— 3.4 Graphen im Speicher Darstellen

- Listen an typed union (enum mit daten)
- Jede union nutzt so viel speicher wie die größte variation benötigt.
- Gute cache Lokalität gegen heap Allocations und pointer

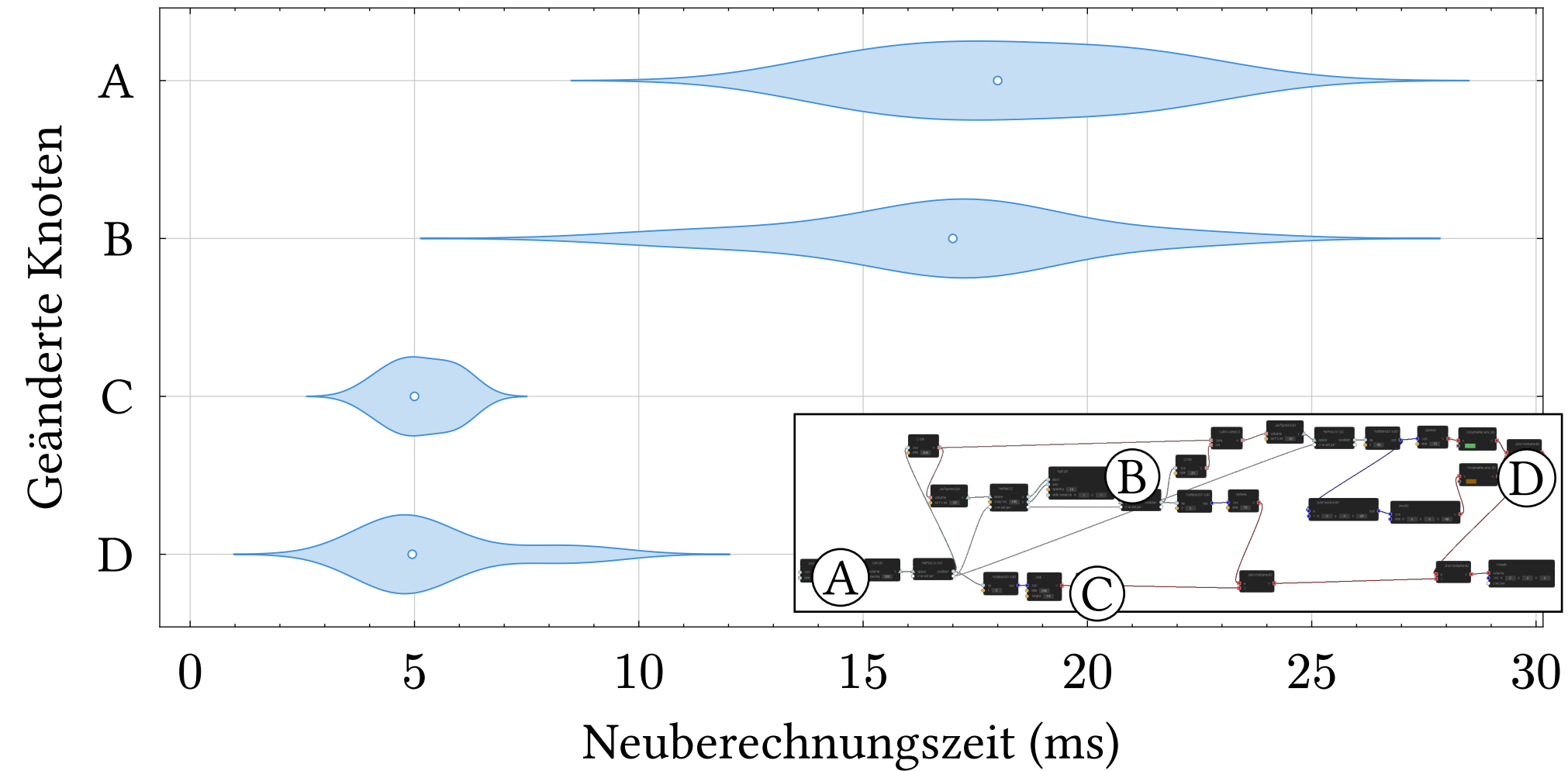
```
fn make_position(child_node: &EditorNode,
                 in_index: usize) -> PosIndex {

    let node = child_node.inputs[in_index];
    let value = match &node.data_type {
        EditorNodeType::Add => {
            let a = make_position(node, 0, ...);
            let b = make_position(node, 1, ...);

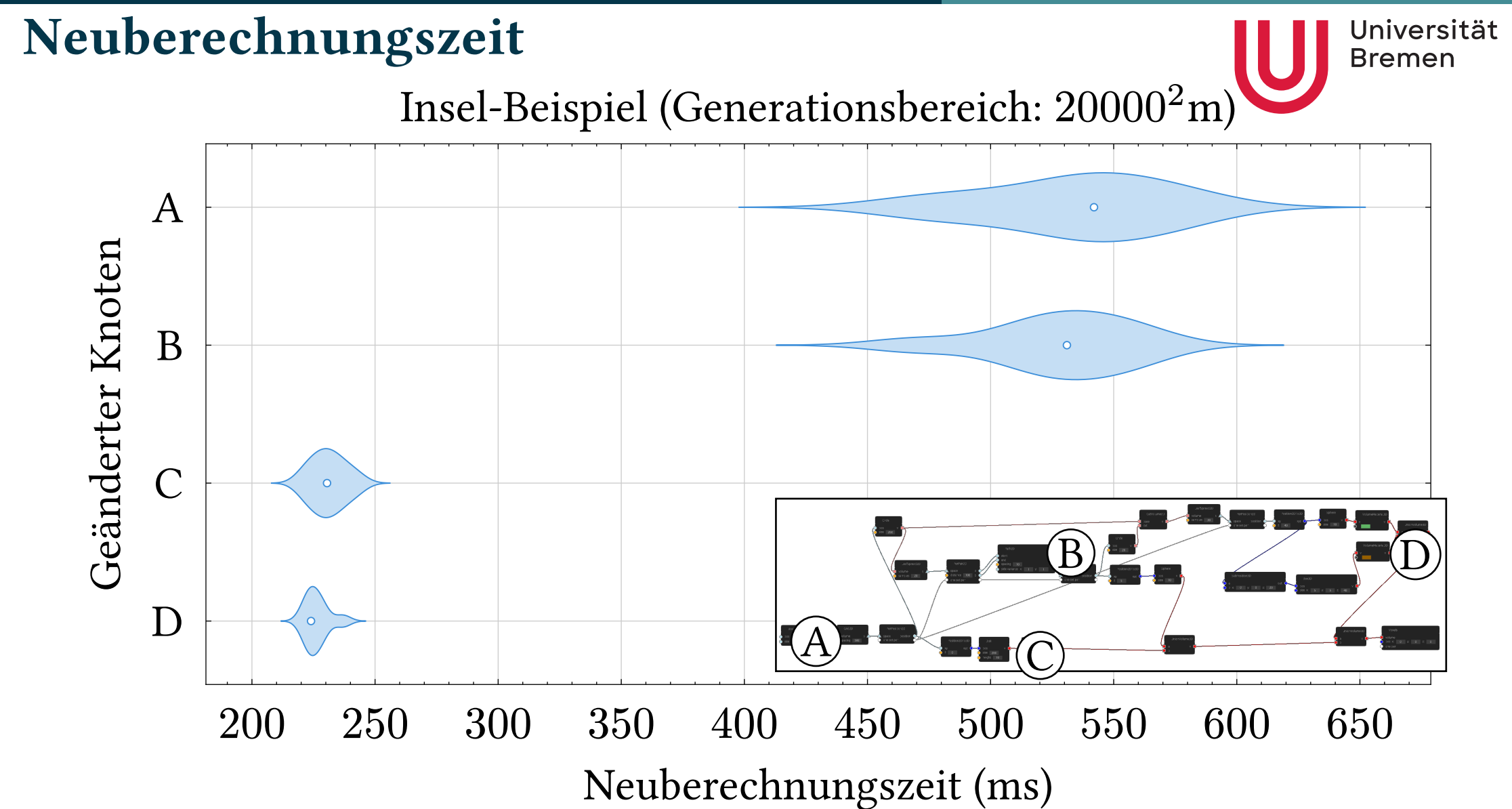
            TemplateValue::Position(PositionValue::Add((a, b)))
        },
        EditorNodeType::Sub => ...
    }
}
```

```
fn calc_position(index: PosIndex) -> SmallVec<Vec3> {  
  
    match values[index] {  
        PositionValue::Add((a_index, b_index)) => {  
            let a = calc_position(a_index);  
            let b = calc_position(b_index);  
  
            return a.cartesian_product(b)  
                .map(|(a_v, b_v)| a_v + b_v);  
        },  
        PositionValue::Sub => ...  
    }  
}
```

Insel-Beispiel (Generationsbereich: 2000^2 m)

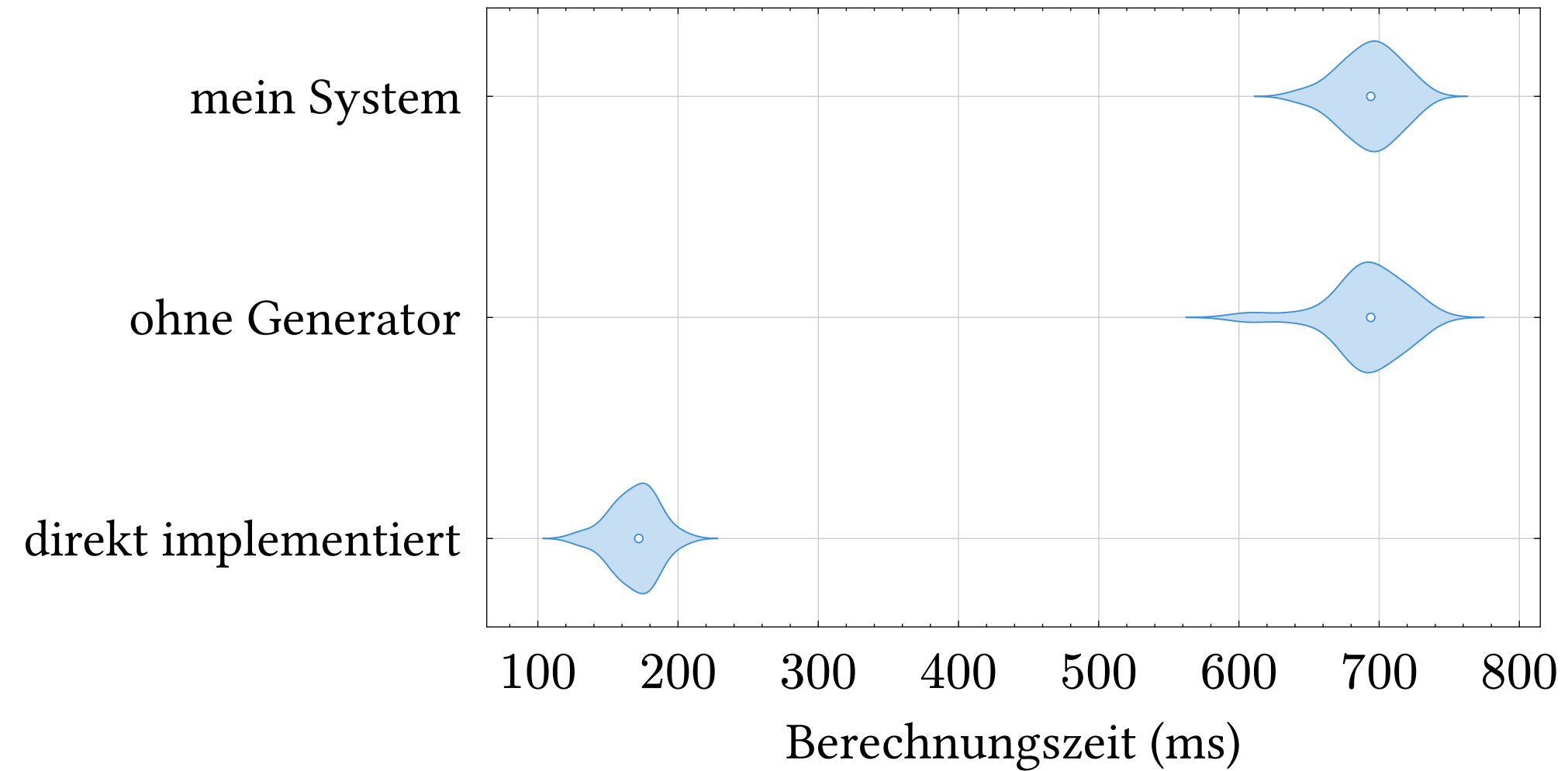


3. Was ich gebaut habe
— 3.7 Neuberechnungszeit

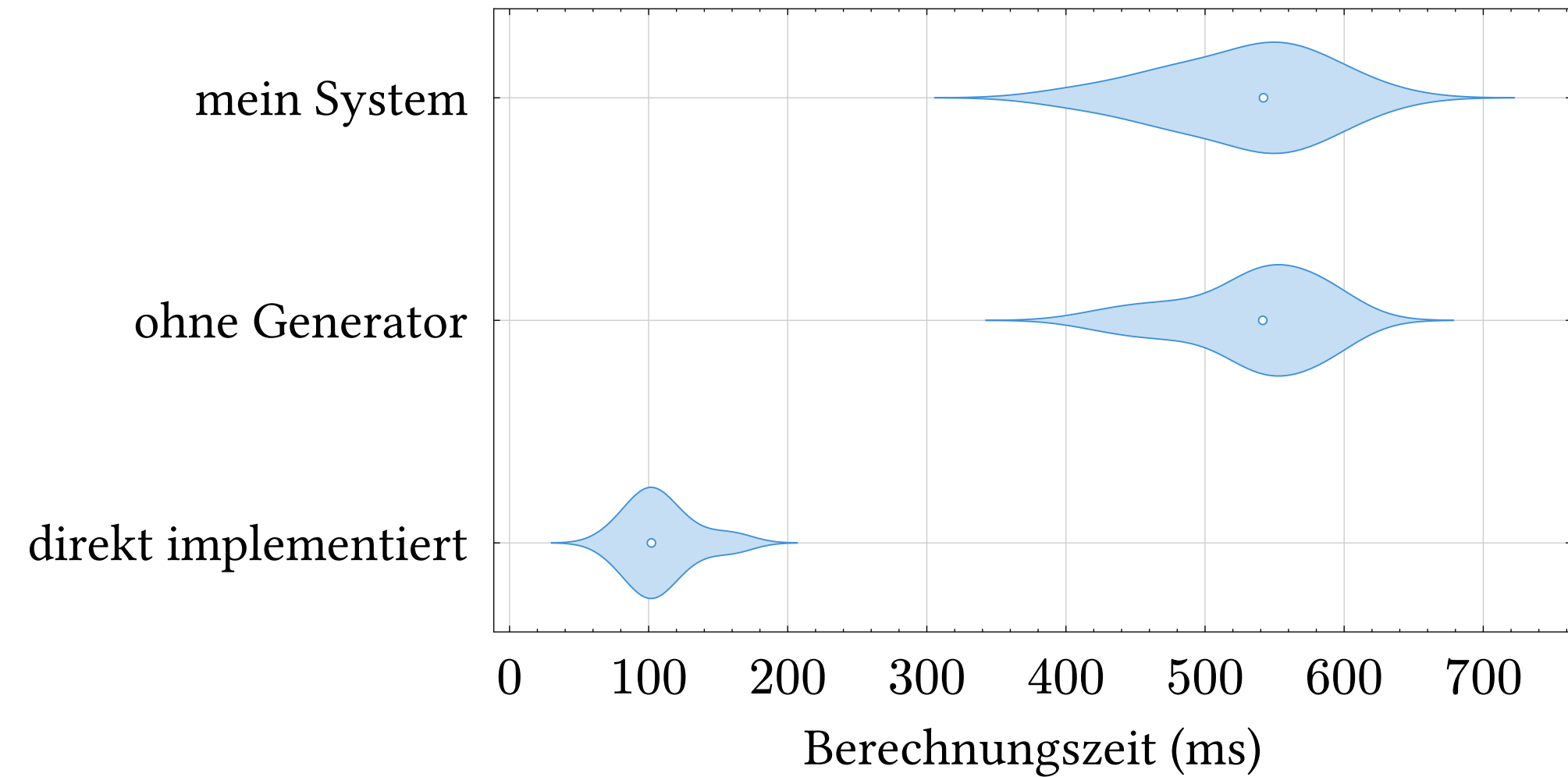


3. Was ich gebaut habe
— 3.7 Neuberechnungszeit

Höhlen-Beispiel (Kreuzungen: 500)



Insel-Beispiel (Bereich: 20000^2m)



3. Was ich gebaut habe
— 3.8 Overhead

Mögliche Fragen

4. Mögliche Fragen

—

Vorteile

- vereinfachte Abstraktionsebene für nicht Programmierer
- schnellere Neugenerien bei großen Welten

4. Mögliche Fragen

— 4.1 Vorteile

Bilder Quellen

- Minecraft: eigener Screenshot



Universität
Bremen

4. Mögliche Fragen

— 4.4 Bilder Quellen